



DOCKER

Complete Preparation Guide

All Topics Covered • 90 Most Important Questions • Detailed Explanation with Commands

90

Questions

10

Topics

100+

Commands

TOPICS COVERED

Topic 1: Docker Basics & Architecture

Topic 2: Docker Images

Topic 3: Dockerfile

Topic 4: Docker Containers

Topic 5: Docker Volumes & Storage

Topic 6: Docker Networking

Topic 7: Docker Compose

Topic 8: Docker Swarm

Topic 9: Docker Security

Topic 10: Advanced & Miscellaneous



ABOUT THE AUTHOR

Sudarshan Gawande - DevOps Engineer | AWS Certified

Passionate DevOps Engineer with expertise in Docker, Kubernetes, CI/CD pipelines, and cloud infrastructure. This guide is a curated collection of the most important Docker questions and answers to learn.

 **LinkedIn**

[sudarshan-gawande](#)

 **GitHub**

[sudarshan-gawande](#)

 **Website**

<https://sudarshangawande.com/>

 **Email**

sudarshangawande98@gmail.com

Table of Contents -

TOPIC 1 | Docker Basics & Architecture

1. What is Docker and why do we use it?
2. What is the difference between Docker and a Virtual Machine (VM)?
3. Explain Docker Architecture in detail.
4. What are the main components of Docker?
5. What is Docker Engine? How does it work?
6. What is Docker Daemon (dockerd)?
7. What is Docker Client?
8. What is Docker Registry?
9. What is the difference between Docker CE and Docker EE?

TOPIC 2 | Docker Images

10. What is a Docker Image?
11. What is the difference between a Docker Image and a Container?
12. What is a Base Image?
13. What is a Dangling Image?
14. How do you create a Docker Image?
15. What is Docker Hub?
16. How do you pull and push an image to Docker Hub?
17. What is the docker commit command?
18. What are Image Layers in Docker?
19. How do you remove unused or dangling Docker images?

TOPIC 3 | Dockerfile

20. What is a Dockerfile?
21. Explain all important Dockerfile instructions: FROM, RUN, CMD, ENTRYPOINT, COPY, ADD, ENV, EXPOSE, WORKDIR, ARG, VOLUME, USER, LABEL, HEALTHCHECK
22. What is the difference between CMD and ENTRYPOINT?
23. What is the difference between COPY and ADD?
24. What is the difference between RUN, CMD, and ENTRYPOINT?
25. What is a Multi-Stage Build in Docker?
26. How do you reduce Docker Image size?
27. What is .dockerignore file?
28. What is the difference between ARG and ENV?

TOPIC 4 | Docker Containers

29. What is a Docker Container?
30. What is the lifecycle of a Docker Container?
31. What is the difference between docker run and docker start?
32. What is the difference between docker stop and docker kill?
33. How do you run a container in detached mode?
34. How do you enter/exec into a running container?
35. How do you pass environment variables to a container?
36. What are restart policies in Docker?
37. How do you limit CPU and Memory of a container?
38. What are namespaces and cgroups in Docker?
39. What is the difference between docker attach and docker exec?
40. How do you copy files between a container and host?

TOPIC 5 | Docker Volumes & Storage

41. What is a Docker Volume?
42. What is the difference between Volume, Bind Mount, and tmpfs Mount?
43. How do you create and use a Docker Volume?
44. How do you share volumes between containers?
45. Where are Docker Volumes stored on the host machine?
46. What is the difference between named volumes and anonymous volumes?
47. How do you backup and restore a Docker Volume?

TOPIC 6 | Docker Networking

48. What is Docker Networking?
49. What are the types of Docker Networks? (bridge, host, none, overlay, macvlan)
50. What is the default network in Docker?
51. What is the difference between Bridge and Host network?
52. How do containers communicate with each other?
53. What is a User-Defined Bridge Network and why is it better?
54. What is an Overlay Network?
55. What is the difference between EXPOSE and -p flag?
56. What is Docker DNS?
57. How do you inspect and troubleshoot Docker networks?

TOPIC 7 | Docker Compose

58. What is Docker Compose?

- 59. What is the difference between Docker and Docker Compose?
- 60. Explain the structure of docker-compose.yml file.
- 61. What are the important Docker Compose commands?
- 62. What is the difference between docker-compose up and docker-compose run?
- 63. How do you scale services in Docker Compose?
- 64. What is depends_on in Docker Compose?
- 65. How do you use environment variables in Docker Compose?
- 66. What are Docker Compose networks and volumes?
- 67. What is the difference between version 2 and version 3 of Docker Compose?

TOPIC 8 | Docker Swarm

- 68. What is Docker Swarm?
- 69. What is the difference between Docker Swarm and Kubernetes?
- 70. What are Manager and Worker nodes in Swarm?
- 71. What is a Docker Service?
- 72. How do you create a Swarm cluster?
- 73. What is Rolling Update in Docker Swarm?
- 74. What is Raft Consensus Algorithm in Swarm?
- 75. How do you deploy a stack in Docker Swarm?

TOPIC 9 | Docker Security

- 76. How do you secure a Docker container?
- 77. What is Docker Content Trust (DCT)?
- 78. What are Docker Secrets?
- 79. Why should you avoid running containers as root?
- 80. What is AppArmor and Seccomp in Docker?
- 81. How do you scan Docker images for vulnerabilities?

TOPIC 10 | Advanced & Miscellaneous

- 82. What is the difference between Docker Image ID and Image Tag?
- 83. What is Docker Context?
- 84. What is BuildKit in Docker?
- 85. How do you monitor Docker containers?
- 86. What is docker inspect and when do you use it?
- 87. What is docker system prune?
- 88. How does Docker handle logging?
- 89. What is the difference between Docker and Podman?
- 90. How do you troubleshoot a Docker container that keeps crashing?

Deep Dive – Explanations & Commands

TOPIC 1 | Docker Basics & Architecture

Q1. What is Docker and why do we use it?

Docker is an open-source containerization platform that allows developers to package applications along with all their dependencies (libraries, configurations, runtime) into a single isolated unit called a Container. This container can run consistently on any machine regardless of the underlying OS or environment.

Why We Use Docker:

- **Consistency:** Eliminates the 'works on my machine' problem — container runs the same in dev, test, and production.
- **Isolation:** Each container runs in its own isolated environment with its own filesystem, network, and process space.
- **Lightweight:** Containers share the host OS kernel — much faster and lighter than Virtual Machines.
- **Portability:** Build once, run anywhere — on your laptop, on-premises servers, or cloud (AWS, GCP, Azure).
- **Faster Deployment:** Containers start in seconds compared to VMs that take minutes.
- **Microservices:** Docker is ideal for deploying independent microservices at scale.
- **CI/CD Integration:** Seamlessly integrates with Jenkins, GitHub Actions, GitLab CI, CircleCI.
- **Resource Efficiency:** Multiple containers can share a single host without wasting resources.

Basic Docker Commands:

```
docker --version      # Check Docker version
docker info           # System-wide Docker info
docker run hello-world # Run first test container
```

Q2. What is the difference between Docker and a Virtual Machine (VM)?

This is one of the most frequently asked Docker interview questions. The core difference is architectural:

Feature	Docker Container	Virtual Machine
OS Kernel	Shares host OS kernel	Has its own full OS kernel
Boot Time	2–5 seconds	1–5 minutes
Size	Megabytes (MBs)	Gigabytes (GBs)

Performance	Near-native (no overhead)	Slower (hypervisor overhead)
Isolation	Process-level (namespaces)	Full hardware-level isolation
Portability	Highly portable	Less portable (large files)
Resource Usage	Very Low	Very High
Startup Speed	Extremely Fast	Slow
Security	Moderate (shared kernel)	High (fully separate OS)
Technology	Linux namespaces + cgroups	Hypervisor (VMware, VirtualBox)

Key Point: VMs virtualize hardware using a Hypervisor. Docker virtualizes at the OS level using Linux Namespaces (for isolation) and Cgroups (for resource control), making it dramatically lighter and faster.

Q3. Explain Docker Architecture in detail.

Docker uses a Client-Server Architecture. The Docker Client communicates with the Docker Daemon, which does the heavy work of building, running, and managing containers.

Components of Docker Architecture:

1. Docker Client (docker CLI)

The primary interface through which users interact with Docker. Commands like `docker run`, `docker build`, `docker pull` are sent from the client to the Docker Daemon via REST API over a Unix socket (`/var/run/docker.sock`) or TCP.

2. Docker Daemon (dockerd)

The background service that does all the actual work. It manages images, containers, networks, and volumes. It listens to Docker API requests and can communicate with other Daemons (in Swarm mode).

3. Docker Registry

Storage and distribution system for Docker images. Docker Hub is the default public registry. Private registries include AWS ECR, Google GCR, Azure ACR, and self-hosted Harbor.

4. Docker Objects

- Images: Read-only templates used to create containers
- Containers: Running instances of images
- Volumes: Persistent storage independent of container lifecycle
- Networks: Enable communication between containers and outside world

Architecture Flow:

```
docker run nginx
|
Docker Client — REST API —> Docker Daemon (dockerd)
                             |
                             Check local image cache
                             |
                             If not found → Pull from Docker Hub
                             |
                             Create Container → Start Container
```

Q4. What are the main components of Docker?

Component	Description
Docker Engine	Core of Docker — includes Docker Daemon, REST API, and CLI
Docker Images	Read-only blueprints/templates for creating containers
Docker Containers	Running instances of Docker images
Dockerfile	Text script with instructions to build a Docker image
Docker Registry	Storage system for images (Docker Hub, ECR, GCR, ACR)
Docker Compose	Tool for multi-container application definition and orchestration
Docker Swarm	Native clustering/orchestration tool for Docker
Docker Volumes	Persistent data storage for containers
Docker Networks	Communication layer between containers

Q5. What is Docker Engine? How does it work?

Docker Engine is the core technology that powers Docker. It is a client-server application made up of three main parts:

- Docker Daemon (dockerd): The server — background service that manages all Docker objects
- REST API: The interface that programs use to communicate with the daemon
- Docker CLI (docker): The command-line client that uses the REST API to communicate with dockerd

When you run `docker run nginx`, the Docker CLI sends an API request to dockerd, which checks the local image cache, pulls the image if needed from Docker Hub, creates a container, and starts it. The daemon manages the entire process.

Check Docker Engine:

```
docker version      # Shows Client and Server versions
docker info         # Detailed engine info (containers, images, storage driver)
```

Q6. What is Docker Daemon (dockerd)?

The Docker Daemon (dockerd) is the persistent background process that manages Docker on a host machine. It is the server side of Docker and handles all Docker operations.

Responsibilities:

- Builds Docker images from Dockerfiles
- Manages full container lifecycle: create, start, stop, remove
- Manages Docker networks, volumes, and storage
- Pulls/pushes images to/from registries
- In Swarm mode: schedules and manages services across nodes

Daemon Commands:

```
sudo systemctl status docker      # Check daemon status
sudo systemctl start docker       # Start daemon
sudo systemctl stop docker        # Stop daemon
sudo systemctl restart docker     # Restart daemon
sudo systemctl enable docker      # Auto-start on boot
journalctl -u docker.service      # View daemon logs

# Daemon config file location
/etc/docker/daemon.json
```

Q7. What is Docker Client?

The Docker Client is the command-line interface (CLI) tool that users interact with. When you type any docker command, the client translates it into an API call and sends it to the Docker Daemon. The client can communicate with both local and remote Docker Daemons.

Common Client Commands:

```
docker run      # Create and start a container
docker build    # Build an image from Dockerfile
docker pull     # Pull image from registry
docker push     # Push image to registry
docker ps       # List containers
docker images   # List images
docker exec     # Execute command in running container
docker logs     # Fetch container logs
docker inspect  # Return detailed info on Docker objects
```

Q8. What is Docker Registry?

A Docker Registry is a storage and distribution server for Docker images. It is where Docker images are stored and retrieved from. Docker Hub is the default public registry, but you can use private registries for security and performance.

Registry Type	Examples	Use Case
Public Registry	Docker Hub, GitHub Container Registry	Open-source, public images
Cloud Private Registry	AWS ECR, Google GCR, Azure ACR	Enterprise cloud deployments
Self-hosted Registry	Harbor, JFrog Artifactory, Nexus	Full control, air-gapped environments

Registry Commands:

```
docker login                # Login to Docker Hub
docker login registry.example.com  # Login to private registry
docker pull nginx:latest    # Pull from Docker Hub
docker pull 123456.dkr.ecr.us-east-1.amazonaws.com/myapp:v1 # Pull from AWS ECR
docker push yourusername/myapp:v1  # Push to Docker Hub
```

Q9. What is the difference between Docker CE and Docker EE?

Feature	Docker CE (Community Edition)	Docker EE (Enterprise Edition)
Cost	Free / Open Source	Paid (subscription-based)
Target	Developers, small teams	Large enterprises
Support	Community support only	Official commercial support from Docker
Security	Basic security features	Advanced security scanning, signing
Management	Basic Docker CLI	Universal Control Plane (UCP) web UI
Registry	Docker Hub (public)	Docker Trusted Registry (DTR)
Updates	Monthly stable + edge releases	Quarterly releases with LTS support
Certification	Not certified	Certified for specific platforms

Note: Docker Inc. rebranded Docker EE to Mirantis Kubernetes Engine (MKE) after Mirantis acquired the enterprise business in 2019.

TOPIC 2 | Docker Images

Q10. What is a Docker Image?

A Docker Image is a lightweight, standalone, read-only template used to create Docker containers. It contains everything needed to run an application: application code, runtime (Node.js, Python, Java), system libraries, environment variables, and configuration files.

Images are built in layers. Each instruction in a Dockerfile creates a new immutable read-only layer. These layers are cached and reused across builds, making Docker very efficient with storage and build times.

Key Docker Image Commands:

```
docker images           # List all local images
docker image ls         # Same as above
docker pull nginx:latest # Pull image from Docker Hub
docker pull ubuntu:22.04 # Pull specific version
docker build -t myapp:v1 . # Build from Dockerfile
docker build -t myapp:v1 -f Dockerfile.prod . # Specify Dockerfile
docker rmi nginx        # Remove an image
docker image rm nginx   # Same as above
docker image prune      # Remove dangling images
docker image prune -a   # Remove ALL unused images
docker image history nginx # Show image layers
docker image inspect nginx # Detailed image metadata
docker tag myapp:v1 myapp:latest # Tag an image
```

Q11. What is the difference between a Docker Image and a Container?

The simplest analogy: Image is like a Class definition in OOP, while a Container is like an Object (instance) created from that class.

Aspect	Docker Image	Docker Container
State	Static, read-only template	Dynamic, running instance with writable layer
Purpose	Blueprint for creating containers	Actual running application process
Mutability	Immutable (cannot be changed)	Mutable — you can write/change files inside
Storage	Stored in registry or local cache	Exists in memory + thin writable layer on disk
Sharing	Shared via Docker Hub or registry	Cannot be directly shared
Lifecycle	Persists until explicitly deleted	Created and deleted frequently
Command (list)	docker images	docker ps (running) / docker ps -a (all)

Q12. What is a Base Image?

A Base Image is the starting point (parent image) for building a Docker image. It is specified with the FROM instruction in a Dockerfile. The base image provides the operating system layer and fundamental tools that your application builds on top of.

Types of Base Images:

- Full OS Images: ubuntu:22.04, debian:11, centos:8 — include full Linux OS tools
- Minimal Images: alpine:3.18 — extremely small (~5MB), uses musl libc and BusyBox
- Language Runtime Images: node:18, python:3.11, openjdk:17 — include OS + language runtime
- Slim Variants: python:3.11-slim — reduced version with fewer packages
- Alpine Variants: node:18-alpine — language runtime on Alpine Linux (smallest)
- Scratch (empty): FROM scratch — completely empty image, used for static binaries

```
FROM ubuntu:22.04      # Full Ubuntu base
FROM alpine:3.18       # Minimal Alpine (5MB)
FROM node:18-alpine    # Node.js on Alpine (smallest Node image)
FROM python:3.11-slim  # Python slim variant
FROM scratch           # Empty base (for Go static binaries)
```

Q13. What is a Dangling Image?

A Dangling Image is a Docker image that has no tag (shows as <none>:<none>) and is not referenced by any container. They appear when you rebuild an image with the same tag — the old image loses its tag and becomes dangling. They waste disk space and should be cleaned up regularly.

Cause of Dangling Images:

```
# First build
docker build -t myapp:latest .      # Image ID: abc123

# Second build (after code change)
docker build -t myapp:latest .      # Image ID: def456 gets the tag
# abc123 now has no tag → becomes DANGLING
```

Handling Dangling Images:

```
# List dangling images
docker images -f dangling=true
docker images -f dangling=true -q  # just IDs

# Remove dangling images
docker image prune

# Remove ALL unused images (not just dangling)
docker image prune -a

# Remove specific dangling image
docker rmi $(docker images -f dangling=true -q)
```

Q14. How do you create a Docker Image?

There are two main ways to create a Docker Image:

Method 1: Using a Dockerfile (Recommended)

```
# Sample Dockerfile for a Python Flask App
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]

# Build the image
docker build -t myflaskapp:v1 .
docker build -t myflaskapp:v1 -f Dockerfile . # explicit Dockerfile path
docker build --no-cache -t myflaskapp:v1 .    # force rebuild without
cache
```

Method 2: Using docker commit (Not Recommended for Production)

```
# Start a base container
docker run -it ubuntu:22.04 bash

# Make changes inside container
apt-get update && apt-get install -y curl
exit

# Commit the container as a new image
docker commit <container_id> mycustom-ubuntu:v1

# Note: Dockerfile approach is always preferred for reproducibility
```

Q15. What is Docker Hub?

Docker Hub is the world's largest public registry for Docker images. It is the default registry that Docker uses when you run `docker pull` without specifying a registry URL. It hosts millions of images from official publishers, verified vendors, and the community.

Key Features of Docker Hub:

- Public repositories: Free unlimited public image hosting
- Private repositories: Limited free private repos, more with paid plans
- Official Images: Verified images from Docker (nginx, ubuntu, mysql, redis, etc.)
- Automated Builds: Connect GitHub/Bitbucket repo to auto-build on push
- Webhooks: Trigger actions when images are pushed
- Docker Scout: Vulnerability scanning for images

Docker Hub Commands:

```
docker login                # Login to Docker Hub
docker search nginx         # Search images on Docker Hub
docker pull nginx:latest    # Pull official nginx image
docker pull ubuntu:22.04    # Pull specific tag

# Push image to Docker Hub
docker tag myapp:v1 username/myapp:v1 # Tag with your username
docker push username/myapp:v1         # Push to Hub
docker logout                        # Logout
```

Q16. How do you pull and push an image to Docker Hub?

Pulling an Image:

```
# Pull latest version
docker pull nginx

# Pull a specific version/tag
docker pull nginx:1.25
docker pull node:18-alpine

# Pull from a private registry
docker pull myregistry.example.com/myimage:tag

# Pull from AWS ECR
docker pull 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
```

Pushing an Image to Docker Hub:

```
# Step 1: Login
docker login

# Step 2: Tag your image with your Docker Hub username
docker tag myapp:v1 yourusername/myapp:v1
docker tag myapp:v1 yourusername/myapp:latest

# Step 3: Push the image
docker push yourusername/myapp:v1
docker push yourusername/myapp:latest

# Step 4: Verify on Docker Hub
# Visit: https://hub.docker.com/r/yourusername/myapp

# Logout after pushing
docker logout
```

Q17. What is the docker commit command?

docker commit creates a new Docker image from the current state of a running or stopped container. It captures all changes made inside the container (installed packages, modified files) and saves them as a new image layer.

Syntax and Example:

```
docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]

# Start a container
docker run -it ubuntu:22.04 bash

# Inside container: install nginx
apt-get update && apt-get install -y nginx
exit

# Get container ID
docker ps -a

# Commit to new image
docker commit abc123xyz ubuntu-with-nginx:v1

# Commit with author and message
docker commit -a "DevTeam" -m "Added nginx" abc123xyz ubuntu-nginx:v1
```

Warning: Using docker commit is not recommended for production. It creates images that are not reproducible or version-controlled. Always prefer Dockerfiles for building images.

Q18. What are Image Layers in Docker?

Docker images are built using a layered architecture. Every instruction in a Dockerfile (FROM, RUN, COPY, ADD, etc.) creates a new read-only layer on top of the previous one. These layers are stacked together to form the final image.

How Layers Work:

- Each layer stores only the CHANGES (diff) from the previous layer — delta compression
- Layers are immutable (read-only) once created
- Layers are cached: unchanged layers are reused across builds, making rebuilds fast
- Multiple images can share the same base layers, saving disk space
- When a container starts, Docker adds a thin writable layer on top (Container Layer)

Layer Example:

```
FROM ubuntu:22.04          # Layer 1: Ubuntu base (~80MB)
RUN apt-get update         # Layer 2: Updated package list
RUN apt-get install -y nginx # Layer 3: nginx installed
COPY ./website /var/www/html # Layer 4: Your website files
CMD ["nginx", "-g", "daemon off;"] # Layer 5: Config (metadata)

# View layers of an image
docker image history nginx
docker inspect nginx | grep -A20 Layers
```

Q19. How do you remove unused Docker images?

Various Ways to Remove Images:

```
# Remove a specific image by name
docker rmi nginx
docker rmi nginx:1.25

# Remove by Image ID
docker rmi abc123def456

# Force remove (even if containers use it)
docker rmi -f nginx

# Remove dangling images (untagged)
docker image prune
docker image prune -f # no confirmation prompt

# Remove ALL unused images (not just dangling)
docker image prune -a

# Remove all images at once (CAUTION!)
docker rmi $(docker images -q)

# Check disk usage before cleanup
docker system df

# Full system cleanup (containers + images + networks)
docker system prune -a --volumes
```

TOPIC 3 | Dockerfile

Q20. What is a Dockerfile?

A Dockerfile is a plain-text script file that contains a sequential set of instructions that tell Docker how to build a Docker Image automatically. Docker reads the Dockerfile from top to bottom and executes each instruction to assemble the final image. Each instruction creates a new layer.

Sample Dockerfile for Node.js Application:

```
# Step 1: Base image
FROM node:18-alpine

# Step 2: Set working directory
WORKDIR /app

# Step 3: Copy package files first (for layer caching)
COPY package*.json ./

# Step 4: Install dependencies
RUN npm ci --only=production

# Step 5: Copy application code
COPY . .

# Step 6: Set environment variable
ENV NODE_ENV=production

# Step 7: Expose port
EXPOSE 3000

# Step 8: Run as non-root user
USER node

# Step 9: Start command
CMD ["node", "server.js"]
```

Build and Run:

```
docker build -t mynodeapp:v1 .          # Build image
docker run -d -p 3000:3000 mynodeapp:v1 # Run container
```

Q21. Explain all important Dockerfile instructions

Instruction	Purpose	Example
FROM	Sets the base/parent image. Must be first instruction.	FROM node:18-alpine
RUN	Executes commands during build. Creates a new layer.	RUN apt-get install -y nginx
CMD	Default command when container starts. Can be overridden.	CMD ["node", "app.js"]
ENTRYPOINT	Main executable. Always runs. Args appended to it.	ENTRYPOINT ["nginx"]
COPY	Copies files/directories from host to container.	COPY ./src /app/src
ADD	Like COPY + supports URLs and auto-extracts archives.	ADD myapp.tar.gz /app/
WORKDIR	Sets working directory for RUN, CMD, COPY, ADD, ENTRYPOINT.	WORKDIR /app
EXPOSE	Documents port the container listens on (metadata only).	EXPOSE 8080
ENV	Sets environment variables available at build + runtime.	ENV DB_HOST=localhost
ARG	Build-time variables (NOT available in running container).	ARG VERSION=1.0
VOLUME	Creates a mount point for external/persistent storage.	VOLUME ["/data"]
USER	Sets the user/UID for running subsequent instructions.	USER appuser
LABEL	Adds key-value metadata to the image.	LABEL maintainer="dev@co.com"
HEALTHCHECK	Defines command to check container health status.	HEALTHCHECK CMD curl -f http://localhost/
ONBUILD	Triggers instruction when image is used as base.	ONBUILD RUN npm install
STOPSIGNAL	Sets system call signal to stop the container.	STOPSIGNAL SIGTERM

Q22. What is the difference between CMD and ENTRYPOINT?

Aspect	CMD	ENTRYPOINT
Purpose	Default command/arguments for container	Main executable that ALWAYS runs

Override	Completely replaced by docker run arguments	NOT replaced — arguments are APPENDED
Multiple	Only the LAST CMD takes effect	Only the LAST ENTRYPOINT takes effect
Shell form	CMD echo hello	ENTRYPOINT echo hello
Exec form	CMD ["echo", "hello"]	ENTRYPOINT ["echo", "hello"]
Best use	Default parameters that may change	Fixed command like a dedicated executable

Practical Examples:

```
# Using CMD (overrideable)
FROM ubuntu
CMD ["echo", "Hello World"]
docker run myimage           # prints: Hello World
docker run myimage echo Goodbye # prints: Goodbye (CMD replaced)

# Using ENTRYPOINT (always runs)
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["Hello World"]          # default arg
docker run myimage           # prints: Hello World
docker run myimage Goodbye    # prints: Goodbye (arg appended)

# Override ENTRYPOINT with --entrypoint flag
docker run --entrypoint /bin/bash myimage
```

Q23. What is the difference between COPY and ADD?

Feature	COPY	ADD
Basic Function	Copies files/dirs from host to container	Same as COPY + extra features
Remote URLs	NOT supported	Supported — downloads from http/https URLs
Archive Extraction	NOT auto-extracted	Auto-extracts .tar, .tar.gz, .tar.bz2, .zip
Predictability	Transparent and predictable	Less predictable due to auto-extract
Best Practice	Preferred for simple file copying	Use only when URL or auto-extraction needed

```
# COPY: Simple and explicit
COPY ./config.json /app/config.json
COPY ./src/ /app/src/

# ADD: Archive extraction
ADD myapp.tar.gz /app/          # Extracts tarball to /app/

# ADD: Remote URL
ADD https://example.com/file.tar.gz /tmp/

# Best practice: Use COPY unless you specifically need ADD features
```

Q24. What is the difference between RUN, CMD, and ENTRYPOINT?

Aspect	RUN	CMD	ENTRYPOINT
When it executes	During docker build (image creation)	When container STARTS	When container STARTS
Creates layer?	Yes — new image layer	No — just metadata	No — just metadata
Purpose	Install packages, run scripts, setup	Default command/args	Main process definition
Can override?	N/A (build-time)	Yes — via docker run args	Needs --entrypoint flag
Multiple?	Yes, each creates a layer	Only last takes effect	Only last takes effect

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y python3  # BUILD-TIME: installs python

ENTRYPOINT ["python3"]      # CONTAINER-START: python3 always runs
CMD ["app.py"]              # CONTAINER-START: default arg for ENTRYPOINT

# docker run myimage          → runs: python3 app.py
# docker run myimage other.py → runs: python3 other.py
```

Q25. What is a Multi-Stage Build in Docker?

Multi-Stage Build is a powerful Dockerfile feature (Docker 17.05+) that uses multiple FROM statements in one Dockerfile. Each FROM starts a new build stage. You can copy artifacts from earlier stages into later ones — leaving behind build tools and dependencies not needed at runtime. This dramatically reduces final image size.

Problem Without Multi-Stage:

A Java application needs JDK + Maven to build, but only the JRE to run. Without multi-stage, you'd include all build tools in the final image unnecessarily, bloating it from ~200MB to ~700MB+.

Example 1 — Go Application:

```
# Stage 1: Build stage
FROM golang:1.21 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o myapp .

# Stage 2: Final stage (tiny!)
FROM alpine:3.18
WORKDIR /root/
COPY --from=builder /app/myapp .
CMD ["/myapp"]
# Result: ~10MB vs ~900MB without multi-stage
```

Example 2 — Java (Maven + JRE):

```
FROM maven:3.9-openjdk-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:resolve
COPY src ./src
RUN mvn package -DskipTests

FROM openjdk:17-jre-slim
WORKDIR /app
COPY --from=build /app/target/myapp.jar .
EXPOSE 8080
CMD ["java", "-jar", "myapp.jar"]
# Result: ~200MB vs ~700MB
```

Q26. How do you reduce Docker Image size?

Best Practices to Reduce Image Size:

1. Use Minimal Base Images:

```
FROM alpine:3.18          # ~5MB
FROM node:18-alpine       # ~50MB vs node:18 (~350MB)
FROM python:3.11-slim     # ~45MB vs python:3.11 (~920MB)
```

2. Use Multi-Stage Builds (most impactful):

```
# Copy only the compiled artifact, not build tools
COPY --from=builder /app/dist ./dist
```

3. Combine RUN Commands to Reduce Layers:

```
# BAD: 3 separate layers
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD: 1 layer, smaller size
RUN apt-get update && \
  apt-get install -y curl && \
  rm -rf /var/lib/apt/lists/*
```

4. Use .dockerignore:

```
node_modules/
.git/
*.log
README.md
tests/
```

5. Avoid Installing Unnecessary Packages:

```
# Add --no-install-recommends to avoid optional packages
RUN apt-get install -y --no-install-recommends curl
```

6. Clean up caches in same RUN layer:

```
RUN pip install --no-cache-dir -r requirements.txt
RUN npm ci --only=production && npm cache clean --force
```

Q27. What is .dockerignore file?

.dockerignore is a file (similar to .gitignore) that specifies files and directories to exclude when building a Docker image. Docker reads this file and excludes matching files from the build context before sending it to the Docker Daemon. This speeds up builds and reduces image size.

Why .dockerignore is Important:

- Reduces build context size — only relevant files are sent to Docker daemon
- Prevents accidentally including secrets (.env files, SSH keys)
- Excludes large directories that aren't needed (node_modules, .git)
- Speeds up docker build significantly for large projects

Example .dockerignore:

```
# Node.js project
node_modules/
npm-debug.log
.npm

# Version control
.git/
.gitignore

# Sensitive files
.env
.env.local
*.pem

# Documentation
README.md
docs/

# Testing
tests/
coverage/

# CI/CD
.github/
Jenkinsfile
```

Q28. What is the difference between ARG and ENV?

Feature	ARG (Build-Time Variable)	ENV (Environment Variable)
Availability	ONLY during docker build	During build AND in running container
In final image	NOT stored in image (except if used in ENV)	Stored in image metadata
Override at build	docker build --build-arg VAR=value	N/A
Override at runtime	Not available	docker run -e VAR=value
Default value	ARG VAR=default	ENV VAR=default
Security	Not in image (safer for tokens)	Visible in docker inspect
Use Case	Build-time config: version, download URL	Runtime config: DB host, port, mode

```
FROM ubuntu:22.04

# Build-time only - pass version number during build
ARG APP_VERSION=1.0.0
ARG BUILD_DATE

# Runtime - available in running container
ENV NODE_ENV=production
ENV PORT=3000

# ARG can set ENV (makes it available at runtime too)
ARG API_KEY
ENV API_KEY=${API_KEY}

RUN echo "Building version: $APP_VERSION"

# Build with ARG override
docker build --build-arg APP_VERSION=2.0.0 --build-arg API_KEY=xyz .

# Runtime ENV override
docker run -e NODE_ENV=development -e PORT=8080 myapp
```

TOPIC 4 | Docker Containers

Q29. What is a Docker Container?

A Docker Container is a lightweight, portable, and isolated runtime environment created from a Docker Image. It is a running (or stopped) instance of an image. Containers include the application code plus everything needed to run it, but share the host OS kernel.

Key Characteristics:

- Isolated: Has its own filesystem, network interfaces, and process space (via namespaces)
- Portable: Run the same container on any machine with Docker installed
- Ephemeral by default: Data is lost when container is removed (use Volumes for persistence)
- Lightweight: Shares host kernel — uses far less resources than VMs
- Fast: Starts in seconds

Container Commands:

```
docker run nginx                # Create and start container
docker run -d nginx             # Detached (background) mode
docker run -d -p 80:80 --name web nginx # Named container with port
docker ps                       # List running containers
docker ps -a                   # List all (including stopped)
docker stop web                 # Graceful stop
docker start web                 # Start stopped container
docker restart web              # Restart
docker rm web                   # Remove stopped container
docker rm -f web                # Force remove running container
```

Q30. What is the lifecycle of a Docker Container?

State	Description	Command to Enter State
Created	Container created but not started yet	docker create <image>
Running	Container is actively executing processes	docker run / docker start
Paused	Container processes are suspended (frozen)	docker pause <container>
Stopped / Exited	Container has stopped executing	docker stop / docker kill
Deleted	Container permanently removed from system	docker rm <container>

Full Lifecycle Commands:

```
# 1. Create without starting
docker create --name myapp nginx

# 2. Start the container
docker start myapp

# 3. OR: Create + Start in one command
docker run -d --name myapp nginx

# 4. Pause container (freeze)
docker pause myapp

# 5. Unpause
docker unpause myapp

# 6. Stop gracefully (SIGTERM → wait 10s → SIGKILL)
docker stop myapp
docker stop --time 30 myapp    # wait 30 seconds before force kill

# 7. Kill immediately (SIGKILL)
docker kill myapp

# 8. Remove stopped container
docker rm myapp
docker rm -f myapp            # Force remove even if running

# Remove all stopped containers
docker container prune
```

Q31. What is the difference between docker run and docker start?

Feature	docker run	docker start
Purpose	Creates a NEW container from an image and starts it	Starts an EXISTING stopped container
Container creation	Always creates a new container	Does NOT create — uses existing
Image required	Yes — specify image name	No — uses container name/ID
Options	Full options (-p, -v, -e, --name, etc.)	Limited options
Use case	First-time container creation	Restarting a stopped container

```
# docker run: creates NEW container
docker run -d -p 80:80 --name webserver nginx

# docker start: starts EXISTING container
docker stop webserver
docker start webserver          # Same container, same config
docker start -ai webserver      # -a = attach, -i = interactive
```

Q32. What is the difference between docker stop and docker kill?

Aspect	docker stop	docker kill
Signal sent	SIGTERM first, then SIGKILL after timeout	SIGKILL immediately (or custom signal)
Graceful shutdown	YES — application can handle the signal and clean up	NO — immediate forceful termination
Timeout	Default 10 seconds before SIGKILL	No timeout
Use case	Production — always prefer for clean shutdown	Emergency — container unresponsive
Custom signal	Limited (SIGTERM)	Any signal: --signal SIGINT, SIGHUP, etc.

```
# Stop gracefully (recommended)
docker stop mycontainer
docker stop --time 30 mycontainer    # Wait 30s before SIGKILL

# Kill forcefully
docker kill mycontainer
docker kill --signal SIGHUP mycontainer    # Custom signal

# Stop all running containers
docker stop $(docker ps -q)
```

Q33. How do you run a container in detached mode?

Detached mode (-d flag) runs a container in the background, freeing up your terminal. The container runs independently and you get back the container ID.

```
# Run in detached (background) mode
docker run -d nginx

# Detached with name and port mapping
docker run -d --name webserver -p 80:80 nginx

# Detached with volume, env var, restart policy
docker run -d \
  --name myapp \
  -p 3000:3000 \
  -v mydata:/app/data \
  -e DB_HOST=localhost \
  --restart unless-stopped \
  myapp:v1

# Check running containers
docker ps
```

```
# Attach to running container's output
docker attach myapp
# View logs of detached container
docker logs -f myapp
```

Q34. How do you enter/exec into a running container?

To get an interactive shell inside a running container, use `docker exec`. This is extremely useful for debugging and troubleshooting.

```
# Get interactive bash shell
docker exec -it mycontainer bash

# Use sh if bash not available (Alpine Linux)
docker exec -it mycontainer sh

# Run specific command without interactive shell
docker exec mycontainer ls -la /app
docker exec mycontainer cat /etc/nginx/nginx.conf
docker exec mycontainer env

# Run as specific user
docker exec -it -u root mycontainer bash
docker exec -it -u 0 mycontainer bash

# Run in specific working directory
docker exec -it -w /app mycontainer bash

# exec flags:
# -i = interactive (keeps STDIN open)
# -t = TTY (allocates pseudo-terminal)
# -u = run as this user
# -w = set working directory
```

Q35. How do you pass environment variables to a container?

Method 1: `-e` flag (single variable):

```
docker run -d -e DB_HOST=localhost -e DB_PORT=5432 myapp
```

Method 2: `--env-file` flag (from file):

```
# Create .env file
DB_HOST=localhost
DB_PORT=5432
DB_USER=admin
DB_PASS=secret123

# Use with Docker
docker run -d --env-file .env myapp
```

Method 3: ENV in Dockerfile (default values):

```
ENV NODE_ENV=production
ENV PORT=3000
```

Method 4: Docker Compose:

```
services:
  app:
    image: myapp
    environment:
      - DB_HOST=database
      - NODE_ENV=production
    env_file:
      - .env
```

View environment variables in container:

```
docker exec mycontainer env
docker inspect mycontainer | grep -A20 'Env'
```

Q36. What are restart policies in Docker?

Policy	Behavior	Use Case
no (default)	Never automatically restart	Development / one-time tasks
on-failure	Restart only on non-zero exit code	Apps that should recover from crashes
on-failure:N	Restart on failure, maximum N times	Retry with limit (e.g., on-failure:5)
always	Always restart regardless of exit status	Production services that must always run
unless-stopped	Always restart UNLESS manually stopped	Production — preferred over 'always'

```
# Set restart policy on run
docker run -d --restart always nginx
docker run -d --restart unless-stopped nginx
docker run -d --restart on-failure:5 myapp

# Update restart policy of existing container
docker update --restart unless-stopped mycontainer
docker update --restart no mycontainer

# Check restart policy
docker inspect mycontainer | grep -i restart
```

Q37. How do you limit CPU and Memory of a container?

Memory Limits:

```
# Limit to 512 MB
docker run -d --memory 512m nginx

# Limit memory and swap (swap = memory + swap total)
docker run -d --memory 512m --memory-swap 1g nginx

# Disable swap (set same as memory)
docker run -d --memory 512m --memory-swap 512m nginx

# Soft limit (reservation)
docker run -d --memory 512m --memory-reservation 256m nginx
```

CPU Limits:

```
# Limit to 0.5 CPU (half a core)
docker run -d --cpus="0.5" nginx

# Limit to 2 CPUs
docker run -d --cpus="2" nginx

# Pin to specific cores (core 0 and 1)
docker run -d --cpuset-cpus="0,1" nginx

# Set CPU shares (relative weight, default 1024)
docker run -d --cpu-shares=512 nginx # gets half the share
```

Monitor Resource Usage:

```
docker stats # Live stats for all containers
docker stats mycontainer # Stats for specific container
docker stats --no-stream # One-time snapshot
```

Q38. What are namespaces and cgroups in Docker?

Linux Namespaces (Isolation):

Namespaces are a Linux kernel feature that provide resource isolation between containers. Docker uses multiple namespaces so each container gets its own view of the system.

Namespace	What It Isolates
PID namespace	Process IDs — each container has its own PID 1 (its main process)
Network namespace	Network interfaces, IP addresses, routing tables, ports
Mount namespace	Filesystem — containers see their own isolated filesystem
UTS namespace	Hostname and domain name (each container can have unique hostname)
IPC namespace	Inter-process communication: shared memory, semaphores, message queues
User namespace	User and group IDs — root in container != root on host

Linux Cgroups (Resource Limiting):

Control Groups (cgroups) is a Linux kernel feature that limits, accounts for, and isolates the resource usage of process groups. Docker uses cgroups to enforce the `--memory` and `--cpus` limits you set.

- Without namespaces: containers could see and interfere with each other's processes
- Without cgroups: one container could consume 100% of CPU and RAM, starving others

Q39. What is the difference between `docker attach` and `docker exec`?

Feature	<code>docker attach</code>	<code>docker exec</code>
What it connects to	Attaches to the container's MAIN process (PID 1)	Runs a NEW separate process inside container
Exit behavior	Exiting (Ctrl+C) can STOP the container	Exiting does NOT stop the container
Use case	Viewing main process output	Debugging, running commands, opening shells
Interactive	Yes, but risky	Yes, preferred for interactive sessions
New process	No — connects to existing PID 1	Yes — starts a new process

```
# docker attach: connect to main process
docker attach mycontainer
# Ctrl+C will STOP the container!
# Ctrl+P Ctrl+Q to detach without stopping

# docker exec: run new process (safer, recommended)
docker exec -it mycontainer bash
# Exiting bash does NOT stop the container
```

Q40. How do you copy files between a container and host?

The `docker cp` command copies files between a running/stopped container and the host machine. This is useful for extracting logs, configs, or other files from containers.

```
# Copy FROM container TO host
docker cp mycontainer:/app/logs/error.log ./error.log
docker cp mycontainer:/etc/nginx/nginx.conf ./nginx.conf

# Copy entire directory FROM container
docker cp mycontainer:/app/logs ./local-logs/

# Copy FROM host TO container
docker cp ./config.json mycontainer:/app/config.json
docker cp ./local-code/ mycontainer:/app/

# Works even on stopped containers!
docker cp stopped-container:/var/log/app.log ./
```

TOPIC 5 | Docker Volumes & Storage

Q41. What is a Docker Volume?

A Docker Volume is a persistent data storage mechanism managed by Docker that exists outside the container's writable layer. By default, data inside a container is lost when the container is deleted. Volumes solve this by storing data on the host filesystem in a location managed by Docker (`/var/lib/docker/volumes/` on Linux), completely independent of the container lifecycle.

Why Use Volumes?

- Data persists even after the container is deleted
- Volumes can be shared and reused between multiple containers
- Volumes are managed by Docker — safer than bind mounts
- Better I/O performance than bind mounts on Mac/Windows
- Volumes can be backed up, restored, and migrated easily

Volume Commands:

```
docker volume create myvolume          # Create named volume
docker volume ls                        # List all volumes
docker volume inspect myvolume         # Detailed info
docker volume rm myvolume              # Remove volume
docker volume prune                     # Remove all unused volumes

# Use volume with container (-v or --mount)
docker run -d -v myvolume:/var/lib/mysql mysql:8

# --mount syntax (more explicit, recommended)
docker run -d --mount type=volume,source=myvolume,target=/data myapp
```

Q42. What is the difference between Volume, Bind Mount, and tmpfs?

Feature	Named Volume	Bind Mount	tmpfs Mount
Managed by	Docker	Host OS (user-specified)	Docker
Host path	<code>/var/lib/docker/volumes/</code>	Any path on host	Host RAM (memory)
Persistence	Persists after container deleted	Depends on host file	Lost when container stops
Performance	Best for containers	Host-dependent	Very fast (in-memory)
Portability	Fully portable	Host path must exist	Not persistent
Use Case	Databases, app data, logs	Dev environment, config injection	Sensitive temp data, caches
Security	More secure	Full host path accessible	Data never written to disk

```
# Named Volume
docker run -d -v mydb-data:/var/lib/mysql mysql
docker run -d --mount type=volume,src=mydb-data,dst=/var/lib/mysql mysql

# Bind Mount (host:container)
docker run -d -v /home/user/code:/app myapp
docker run -d -v $(pwd):/app myapp # current dir

# Read-only Bind Mount
docker run -d -v /host/config:/app/config:ro myapp

# tmpfs Mount
docker run -d --tmpfs /tmp myapp
docker run -d --tmpfs /tmp:size=100m myapp # limit size
```

Q43. How do you create and use a Docker Volume?

```
# Create a named volume
docker volume create pgdata

# Run PostgreSQL with persistent data volume
docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:15

# Inspect volume location on host
docker volume inspect pgdata
# 'Mountpoint': '/var/lib/docker/volumes/pgdata/_data'

# Multiple containers sharing same volume
docker run -d --name app1 -v shared-data:/data myapp
docker run -d --name app2 -v shared-data:/data myapp
```

Q44. How do you share volumes between containers?

Multiple containers can mount the same named volume, making it a shared data space. This is useful for log aggregators, data processing pipelines, and shared configuration.

```
# Create shared volume
docker volume create shared-logs

# Container 1: App writes logs
docker run -d --name app \
  -v shared-logs:/app/logs \
  myapp:latest
```

```
# Container 2: Log aggregator reads logs
docker run -d --name log-watcher \
  -v shared-logs:/logs:ro \
  busybox tail -f /logs/app.log

# Using --volumes-from to share all volumes from another container
docker run -d --name app -v mydata:/data myapp
docker run -it --volumes-from app ubuntu bash
# Now ubuntu can see /data from myapp's volume
```

Q45. Where are Docker Volumes stored on the host machine?

Platform	Volume Storage Location
Linux	/var/lib/docker/volumes/<volume-name>/_data
Mac (Docker Desktop)	Inside the Docker Desktop VM: /var/lib/docker/volumes/ (not directly accessible from Mac)
Windows (Docker Desktop)	Inside the WSL2 VM or Hyper-V VM (not directly accessible from Windows host)

```
# Find exact location of a volume
docker volume inspect myvolume

# Output includes:
# "Mountpoint": "/var/lib/docker/volumes/myvolume/_data"

# Access on Linux directly
sudo ls /var/lib/docker/volumes/myvolume/_data
```

Q46. What is the difference between named volumes and anonymous volumes?

Feature	Named Volume	Anonymous Volume
Name	You provide a name: -v mydata:/path	Docker generates a random ID: -v /path
Reusability	Easy to reference and reuse	Hard to reference — random UUID
Persistence	Persists and manageable	Persists but hard to manage
Cleanup	Must explicitly remove	Removed with docker rm -v
docker volume ls	Shows with name	Shows with random UUID
Recommended	YES — for production use	Avoid — mostly for temporary use

```
# Named volume (recommended)
docker run -d -v pgdata:/var/lib/postgresql/data postgres
# Volume name: pgdata

# Anonymous volume
docker run -d -v /var/lib/postgresql/data postgres
# Volume name: 8f3a9b2c4d1e... (random UUID)

# Also in Dockerfile
VOLUME ["/data"] # Creates anonymous volume if not overridden
```

Q47. How do you backup and restore a Docker Volume?

Backup a Volume:

```
# Method 1: Run a temporary container to create archive
docker run --rm \
  -v myvolume:/source:ro \
  -v $(pwd):/backup \
  ubuntu tar czf /backup/myvolume_backup.tar.gz -C /source .
```

Restore a Volume:

```
# Create new target volume
docker volume create myvolume-restored

# Extract backup into new volume
docker run --rm \
  -v myvolume-restored:/target \
  -v $(pwd):/backup:ro \
  ubuntu tar xzf /backup/myvolume_backup.tar.gz -C /target
```

Copy Volume to Another Host:

```
# On source host: backup
docker run --rm -v myvolume:/data -v $(pwd):/backup ubuntu \
  tar czf /backup/myvolume.tar.gz -C /data .

# Transfer to another host
scp myvolume.tar.gz user@otherhost:/tmp/

# On target host: restore
docker volume create myvolume
docker run --rm -v myvolume:/data -v /tmp:/backup ubuntu \
  tar xzf /backup/myvolume.tar.gz -C /data
```

TOPIC 6 | Docker Networking

Q48. What is Docker Networking?

Docker Networking is the system that enables containers to communicate with each other, with the host machine, and with external networks. Docker provides several networking drivers (bridge, host, none, overlay, macvlan) to support different use cases from single-host development to multi-host production deployments.

Core Networking Commands:

```
docker network ls          # List all networks
docker network create mynetwork # Create bridge network
docker network inspect mynetwork # Inspect network details
docker network connect mynetwork myapp # Connect container to network
docker network disconnect mynetwork myapp # Disconnect container
docker network rm mynetwork # Remove network
docker network prune       # Remove unused networks
```

Q49. What are the types of Docker Networks?

Network Driver	Description	Use Case
bridge	Default driver. Creates private internal network. Containers get own IP. NAT for external access.	Single-host container communication (default)
host	Container shares host's network stack. No isolation. Uses host's ports directly.	High-performance apps needing low latency networking
none	Container has no network access whatsoever. Complete network isolation.	Maximum isolation, batch jobs, security-sensitive tasks
overlay	Multi-host network. Connects Docker Swarm nodes. Encrypts traffic between hosts.	Docker Swarm, distributed microservices
macvlan	Assigns real MAC address to container. Appears as physical device on network.	Legacy applications needing direct network access

```
# Create different network types
docker network create --driver bridge mybridge
docker network create --driver overlay myoverlay

# Run with host network (no -p needed)
docker run -d --network host nginx
# nginx listens on host port 80 directly

# Run with no network
docker run -d --network none myapp
```

Q50. What is the default network in Docker?

When you run a container without specifying a network, Docker automatically connects it to the default bridge network called 'bridge'. All containers on the default bridge network can communicate with each other via IP address, but NOT via container name (DNS resolution is not available on the default bridge).

```
# View default networks
docker network ls
# NETWORK ID      NAME      DRIVER      SCOPE
# alb2c3d4e5f6    bridge    bridge       local    ← default
# b2c3d4e5f6a1    host      host         local
# c3d4e5f6a1b2    none      null         local

# Inspect default bridge network
docker network inspect bridge

# All containers get an IP from 172.17.0.0/16 by default
```

Q51. What is the difference between Bridge and Host network?

Feature	Bridge Network	Host Network
Network isolation	YES — container has own network namespace	NO — shares host's network namespace
Port mapping	Required: -p 8080:80	Not needed — container uses host ports directly
Performance	Slight overhead due to NAT	Best possible — no NAT overhead
Container IP	Container gets own IP (e.g., 172.17.0.2)	Uses host's IP address
Port conflicts	Less likely — containers isolated	Possible — container uses host ports
Security	More secure (isolation)	Less secure (shares host network)
DNS resolution	Yes (on user-defined bridge)	Uses host's DNS

Q52. How do containers communicate with each other?

Method 1: User-Defined Bridge Network (Recommended):

```
# Create custom network
docker network create app-network

# Run containers on same network
docker run -d --name backend --network app-network mybackend
docker run -d --name frontend --network app-network myfrontend

# frontend can now reach backend by NAME: http://backend:5000
# Docker's embedded DNS resolves container names automatically
```

Method 2: Default Bridge Network (limited):

```
# Containers can only communicate by IP, not by name
docker run -d --name app1 myapp
docker run -d --name app2 myapp
# Must use docker inspect app1 to find IP, then use that IP
```

Method 3: Docker Compose (automatic):

```
# All services in same compose file automatically get a shared network
# Services can reference each other by service name
services:
  frontend:
    environment:
      - API_URL=http://backend:5000 # 'backend' is DNS name
  backend:
    image: mybackend
```

Q53. What is a User-Defined Bridge Network and why is it better?

A User-Defined Bridge Network is a custom network you create with `docker network create`. It is significantly better than the default bridge network for the following reasons:

Feature	Default Bridge	User-Defined Bridge
DNS / Name Resolution	NOT available — only IP	YES — containers communicate by name
Isolation	All containers share one bridge	Better isolation per application
Configuration	Fixed, Docker-managed	Customizable (subnet, IP range, etc.)
On-demand connect	Cannot add containers after creation	Can connect/disconnect any time
Environment vars	Shared via <code>--link</code> (deprecated)	Clean DNS-based discovery

```
# Create user-defined network with custom subnet
docker network create \
  --driver bridge \
  --subnet 192.168.10.0/24 \
  --gateway 192.168.10.1 \
  my-app-network

# Containers communicate by name
docker run -d --name db --network my-app-network mysql
docker run -d --name api --network my-app-network myapi
# api can connect to mysql at: db:3306
```

Q54. What is an Overlay Network?

An Overlay Network is a Docker network driver that enables multi-host container communication. It creates a distributed network among multiple Docker hosts in a Swarm cluster, allowing containers running on different hosts to communicate securely as if they were on the same local network.

- Built on top of host networks using VXLAN encapsulation
- Traffic between nodes is encrypted (optional, with `--opt encrypted`)
- Requires Docker Swarm mode to be active
- Automatically handles routing between hosts

```
# Initialize swarm first
docker swarm init

# Create overlay network
docker network create --driver overlay --attachable my-overlay

# Create overlay with encryption
docker network create --driver overlay --opt encrypted my-secure-overlay

# Deploy service on overlay network
docker service create --network my-overlay --name api myapi
docker service create --network my-overlay --name db mysql
# api can reach db by name across different Swarm nodes
```



Q55. What is the difference between EXPOSE and -p flag?

Feature	EXPOSE (Dockerfile)	-p flag (docker run)
Type	Documentation / metadata only	Actual runtime port mapping
Effect	Documents which port app listens on. Does NOT open/bind the port.	Actually maps host_port:container_port
Accessible from host	NO — not accessible from outside Docker	YES — accessible from host and outside
Accessible from containers	YES — other containers can reach it	YES
Purpose	Documents intent, enables -P auto-publish	Makes service accessible externally

```
# In Dockerfile - just documentation
EXPOSE 8080

# -p: Maps host port to container port
docker run -d -p 80:8080 myapp # host:80 → container:8080
docker run -d -p 8080:8080 myapp # same port on both
docker run -d -p 0.0.0.0:80:8080 myapp # all interfaces
docker run -d -p 127.0.0.1:80:8080 myapp # localhost only

# -P (uppercase): publish ALL EXPOSE'd ports to random host ports
docker run -d -P myapp
docker port myapp # see auto-assigned ports
```

Q56. What is Docker DNS?

Docker has a built-in embedded DNS server (127.0.0.11) that provides automatic DNS resolution for containers on user-defined networks. Containers can resolve each other's names to IP addresses automatically, without any manual configuration.

- Only works on user-defined bridge, overlay, and macvlan networks
- Does NOT work on the default bridge network
- Container name = DNS hostname
- In Docker Compose, service name = DNS hostname

```
docker network create mynet
docker run -d --name database --network mynet postgres
docker run -d --name webapp --network mynet myapp

# Inside webapp container, 'database' resolves to the postgres container's IP
docker exec -it webapp ping database
docker exec -it webapp curl database:5432
```

Q57. How do you inspect and troubleshoot Docker networks?

```
# View all networks
docker network ls

# Inspect a network (see connected containers, IPs, subnet)
docker network inspect bridge
docker network inspect mynetwork

# Get container's IP address
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
mycontainer

# Check container's network settings
docker inspect mycontainer | grep -A 30 NetworkSettings

# Test connectivity between containers
docker exec -it appl ping app2
docker exec -it appl curl http://app2:5000/health

# Check port bindings
docker port mycontainer

# View network traffic (install net-tools in container)
docker exec mycontainer netstat -tlnp
```

TOPIC 7 | Docker Compose

Q58. What is Docker Compose?

Docker Compose is a tool for defining and running multi-container Docker applications. You use a single YAML file (docker-compose.yml) to configure all your application services, networks, and volumes. Then, with a single command (docker-compose up), you create and start all services defined in your configuration.

- Manages multi-container applications as a single unit
- Defines relationships and dependencies between services
- Creates networks and volumes automatically
- Supports environment-specific configurations via .env files
- Ideal for development, testing, and CI environments

Install Docker Compose:

```
# Docker Compose V2 is now built into Docker CLI
docker compose version

# Older V1 (standalone)
docker-compose --version
```

Q59. Explain the structure of docker-compose.yml file.

The docker-compose.yml file has a well-defined structure with top-level keys for version, services, networks, and volumes.

```
version: '3.8'           # Compose file format version

services:                # Define all containers
  web:                   # Service name (acts as DNS hostname)
    image: nginx:latest  # Use existing image
    # OR
    build:                # Build from Dockerfile
      context: .
      dockerfile: Dockerfile
    ports:                # Port mapping
      - "80:80"
    volumes:              # Volumes
      - ./html:/usr/share/nginx/html
      - appdata:/data
    environment:          # Environment variables
      - NODE_ENV=production
      - DB_HOST=database
    env_file:             # Load from file
      - .env
    depends_on:           # Start order
```

```

- database
networks:          # Connect to networks
- app-net
restart: unless-stopped
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost"]
  interval: 30s
  timeout: 10s
  retries: 3

database:
  image: postgres:15
  volumes:
    - pgdata:/var/lib/postgresql/data
  environment:
    POSTGRES_DB: mydb
    POSTGRES_USER: admin
    POSTGRES_PASSWORD_FILE: /run/secrets/db_password
  networks:
    - app-net

networks:          # Define custom networks
  app-net:
    driver: bridge

volumes:          # Define named volumes
  pgdata:
  appdata:

```

Q60. What are the important Docker Compose commands?

Command	Description
docker-compose up	Create and start all services
docker-compose up -d	Start in detached (background) mode
docker-compose up --build	Force rebuild images before starting
docker-compose up --scale web=3	Start 3 instances of web service
docker-compose down	Stop and remove containers and networks
docker-compose down -v	Also remove named volumes
docker-compose down --rmi all	Also remove images
docker-compose ps	List running service containers
docker-compose logs	View all service logs
docker-compose logs -f api	Follow logs of specific service
docker-compose exec api bash	Shell into running service container
docker-compose run api pytest	Run one-off command in new container
docker-compose build	Build or rebuild service images

<code>docker-compose build --no-cache</code>	Build without cache
<code>docker-compose stop</code>	Stop services (containers remain)
<code>docker-compose start</code>	Start stopped services
<code>docker-compose restart</code>	Restart all services
<code>docker-compose pull</code>	Pull latest images for all services
<code>docker-compose config</code>	Validate and view the effective config

Q61. What is the difference between `docker-compose up` and `docker-compose run`?

Feature	<code>docker-compose up</code>	<code>docker-compose run</code>
Purpose	Start ALL services defined in compose file	Run a ONE-OFF command in a service container
Dependencies	Starts all <code>depends_on</code> services	Starts <code>depends_on</code> services by default
Port mapping	Maps ports as defined in compose file	Does NOT map ports by default
Cleanup	Containers persist after stop	Container removed after run (with <code>--rm</code>)
Use case	Start the whole application	Run migrations, tests, bash, admin tasks

```
# Start entire application
docker-compose up -d

# Run a one-off task (database migration)
docker-compose run --rm api python manage.py migrate

# Run tests
docker-compose run --rm api pytest tests/

# Open shell in a service
docker-compose run --rm api bash
```

Q62. How do you scale services in Docker Compose?

```
# Scale web service to 3 replicas
docker-compose up -d --scale web=3

# Scale multiple services
docker-compose up -d --scale web=3 --scale worker=5
```

```
# Check scaled containers
docker-compose ps

# Note: When scaling, avoid fixed port mapping
# This will FAIL when scaling (port conflict)
ports:
  - "80:80"    # ← only 1 container can bind port 80

# Use a load balancer like nginx or traefik instead:
ports:
  - "80"        # ← Docker assigns random host ports
```

Q63. What is depends_on in Docker Compose?

depends_on specifies service startup and shutdown dependency order. It tells Docker Compose that one service depends on another and should start only after that service's container has started. IMPORTANT: depends_on does NOT wait for a service to be 'ready' — only for the container to start.

```
# Basic depends_on (container started, not necessarily ready)
services:
  app:
    depends_on:
      - db
      - redis

# Advanced: wait for health check (Compose v3.9+)
services:
  app:
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started

  db:
    image: postgres:15
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
```

Q64. How do you use environment variables in Docker Compose?

Method 1: environment key directly in compose file:

```
services:
  app:
    environment:
      NODE_ENV: production
      DB_HOST: database
      DB_PORT: 5432
```

Method 2: env_file key (read from file):

```
services:
  app:
    env_file:
      - .env
      - .env.production
```

Method 3: .env file for Compose variable substitution:

```
# .env file (in same directory as docker-compose.yml)
IMAGE_TAG=1.5.0
DB_PASSWORD=secret123

# docker-compose.yml - uses ${VAR} substitution
services:
  app:
    image: myapp:${IMAGE_TAG}
    environment:
      - DB_PASSWORD=${DB_PASSWORD}
```

Method 4: Shell environment variables:

```
export TAG=2.0.0
docker-compose up -d
# ${TAG} in compose file resolves to '2.0.0'
```

Q65. What are Docker Compose networks and volumes?

Networks in Docker Compose:

By default, Docker Compose creates a single default network for all services. All services can reach each other by service name. You can also define custom networks for more control.

```
networks:
  frontend-net:      # Services on this can talk to each other
    driver: bridge
  backend-net:       # Database isolated from frontend
    driver: bridge
    internal: true   # No external internet access

services:
  nginx:
```

```

networks: [frontend-net, backend-net] # Bridge between both
api:
  networks: [backend-net]
database:
  networks: [backend-net]

```

Volumes in Docker Compose:

```

volumes:
  pgdata: # Docker manages this
  redisdata:
  appconfig: # External volume (must exist)
    external: true

services:
  postgres:
    volumes:
      - pgdata:/var/lib/postgresql/data # named volume
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql # bind mount

```

Q66. What is the difference between docker-compose up and docker-compose run?

(See Q61 above for full comparison.) The key distinction:

- docker-compose up: Starts ALL services defined in the file (the whole application stack)
- docker-compose run: Starts a single service and runs a one-time command, then exits
- docker-compose exec: Runs a command in an ALREADY running service container

Q67. What is the difference between Compose version 2 and version 3?

Feature	Version 2	Version 3
Target	Single-host Docker Compose	Docker Swarm + Compose
deploy key	Not supported	Supported (for Swarm services)
Resources (CPU/Memory)	In root level	Under deploy.resources
depends_on with condition	Limited support	Full condition support
volumes_from	Supported	Not supported
links	Supported (deprecated)	Not recommended
Recommendation	For Compose-only deployments	For Swarm or modern Compose v2+ CLI

Note: As of Docker Compose v2 CLI (2022+), the version key is optional/deprecated. Modern Compose files work without specifying a version.

TOPIC 8 | Docker Swarm

Q68. What is Docker Swarm?

Docker Swarm is Docker's native clustering and container orchestration tool. It allows you to create and manage a cluster of Docker hosts (called a Swarm) and deploy services across them. The Swarm ensures your services run reliably across multiple nodes, automatically handling failover and load distribution.

Key Swarm Concepts:

Concept	Description
Swarm	A cluster of Docker nodes (hosts) running Docker in Swarm mode
Node	A Docker host participating in the Swarm
Manager Node	Controls and orchestrates the Swarm. Maintains cluster state. Dispatches tasks.
Worker Node	Executes containers/tasks assigned by Manager nodes
Service	The definition of a task to run — includes image, replicas, ports, and more
Task	A single container instance running as part of a service
Replica	Number of identical task instances for a service
Stack	Group of interrelated services defined in a Compose file, deployed to Swarm

Quick Swarm Setup:

```
# On first manager node
docker swarm init --advertise-addr 192.168.1.100

# Get join token for workers
docker swarm join-token worker

# Run on worker nodes
docker swarm join --token SWMTKN-xxx 192.168.1.100:2377

# View cluster nodes
docker node ls
```

Q69. What is the difference between Docker Swarm and Kubernetes?

Feature	Docker Swarm	Kubernetes (K8s)
Setup	Simple — 3 commands	Complex — steep learning curve
Scalability	Good for smaller clusters	Excellent — thousands of nodes

Auto-scaling	Manual only	Auto (HPA — Horizontal Pod Autoscaler)
Load Balancing	Built-in basic LB	Advanced, multiple options
Health checks	Basic service health	Liveness, readiness, startup probes
Rolling updates	Supported	Advanced with strategies
Community/Ecosystem	Smaller, declining	Massive — industry standard
Dashboard	Portainer (3rd party)	Kubernetes Dashboard, Lens, Rancher
Storage	Volumes + plugins	Rich PV/PVC storage abstraction
Networking	Overlay network	CNI plugins (Calico, Flannel, etc.)
Learning curve	Easy	Steep
Current adoption	Declining in favor of K8s	Dominant in production

Q70. What are Manager and Worker nodes in Swarm?

Manager Nodes:

- Maintain the cluster state and configuration (using Raft consensus)
- Schedule tasks (containers) to worker nodes
- Accept docker service and docker stack commands
- Can also run tasks like workers (configurable)
- Recommended: 3, 5, or 7 managers for high availability (odd numbers for Raft quorum)

Worker Nodes:

- Execute the tasks (containers) assigned by manager nodes
- Report task status back to managers
- Cannot accept Swarm management commands
- Can be promoted to manager or demoted to worker

```
# View all nodes and roles
docker node ls

# Promote worker to manager
docker node promote worker-node-1

# Demote manager to worker
docker node demote manager-node-2

# Drain a node (stop tasks, maintenance)
docker node update --availability drain worker-node-1

# Inspect a node
docker node inspect worker-node-1 --pretty
```

Q71. What is a Docker Service?

A Docker Service is the definition of how to run containers at scale in Docker Swarm mode. It specifies the image to use, how many replicas (instances) to run, port mappings, network, volumes, update policy, and more. Swarm automatically schedules tasks (container instances) across available nodes.

```
# Create a service with 3 replicas
docker service create \
  --name webapp \
  --replicas 3 \
  --publish published=80,target=8080 \
  --network my-overlay \
  myapp:latest

# List services
docker service ls

# View service tasks (replicas)
docker service ps webapp

# Scale service
docker service scale webapp=5

# Update service image
docker service update --image myapp:v2 webapp

# Remove service
docker service rm webapp

# View service logs
docker service logs -f webapp
```

Q72. How do you create a Swarm cluster?

```
# — Step 1: Initialize Swarm on Manager node —
docker swarm init --advertise-addr <MANAGER-IP>
# Output shows join token for workers

# — Step 2: Get join tokens —
docker swarm join-token worker    # Token for worker nodes
docker swarm join-token manager   # Token for additional managers

# — Step 3: Join worker nodes —
# Run this on each worker node:
docker swarm join \
  --token SWMTKN-1-xxxxxxx \
  192.168.1.100:2377

# — Step 4: Verify cluster —
docker node ls

# — Step 5: Deploy a service —
docker service create --name nginx --replicas 3 -p 80:80 nginx

# — Step 6: Check service health —
docker service ps nginx

# Leave the swarm
docker swarm leave                # On worker node
docker swarm leave --force         # On manager node (destroys swarm)
```

Q73. What is Rolling Update in Docker Swarm?

Rolling Update is a deployment strategy in Docker Swarm that updates service replicas gradually — updating a few containers at a time — rather than all at once. This ensures zero downtime during updates.

Rolling Update Process:

- Stop a batch of old task replicas
- Start new task replicas with the updated image/config
- Wait (delay) before proceeding to next batch
- Continue until all replicas are updated

```
# Create service with rolling update policy
docker service create \
  --name webapp \
  --replicas 6 \
  --update-parallelism 2 \      # Update 2 replicas at a time
  --update-delay 10s \         # Wait 10s between batches
  --update-failure-action rollback \ # Rollback on failure
  myapp:v1

# Perform rolling update
docker service update --image myapp:v2 webapp

# Monitor update progress
docker service ps webapp

# Rollback to previous version
docker service rollback webapp
```

Q74. What is Raft Consensus Algorithm in Swarm?

Docker Swarm uses the Raft Consensus Algorithm to maintain a consistent state across all Manager nodes. Raft ensures all managers agree on the current state of the cluster — services, tasks, node membership — even in the event of manager failures.

Key Points:

- All Manager nodes maintain a copy of the cluster state
- One Manager is elected as the Leader — it makes all scheduling decisions
- If the Leader fails, a new Leader is elected automatically
- For Raft to function, a quorum (majority) of managers must be available
- Quorum = $(N/2) + 1$ where N = total managers

Total Managers	Quorum (Minimum)	Fault Tolerance (Max failures)
1	1	0
3	2	1
5	3	2
7	4	3

Best Practice: Always use an odd number of managers (3, 5, 7) for fault tolerance. If you have 4 managers and 2 fail, you lose quorum. With 5 managers, you can tolerate 2 failures.

Q75. How do you deploy a stack in Docker Swarm?

A Docker Stack is a group of interrelated services defined in a Docker Compose-format file and deployed to a Swarm cluster as a single unit. It is the production-level equivalent of docker-compose up.

```
# docker-stack.yml (similar to docker-compose.yml)
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    deploy:                                # Swarm-specific deployment config
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
      restart_policy:
        condition: on-failure
      resources:
        limits:
          cpus: '0.5'
          memory: 256M
    networks:
      - webnet

networks:
  webnet:

# Deploy the stack
docker stack deploy -c docker-stack.yml mystack

# List stacks
docker stack ls

# List services in a stack
docker stack services mystack

# List tasks in a stack
docker stack ps mystack

# Remove a stack
docker stack rm mystack
```

TOPIC 9 | Docker Security

Q76. How do you secure a Docker container?

1. Run as Non-Root User:

```
# In Dockerfile
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser

# Or at runtime
docker run -u 1001:1001 myapp
```

2. Use Read-Only Root Filesystem:

```
docker run --read-only myapp
# If app needs to write: add --tmpfs for specific dirs
docker run --read-only --tmpfs /tmp myapp
```

3. Drop Linux Capabilities:

```
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp
```

4. Use Minimal Base Images:

- Use alpine, distroless, or scratch for smallest attack surface
- Fewer packages = fewer vulnerabilities

5. Scan for Vulnerabilities:

```
docker scout cves myimage:latest
trivy image myimage:latest
```

6. Use Secrets (not ENV) for Credentials:

```
docker secret create dbpass ./password.txt
docker service create --secret dbpass myapp
```

7. Set Resource Limits:

```
docker run --memory 512m --cpus 1 myapp
```

8. Disable inter-container communication:

```
# In daemon.json
{ "icc": false }
```

9. Run Docker Bench Security Audit:

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  docker/docker-bench-security
```

Q77. What is Docker Content Trust (DCT)?

Docker Content Trust (DCT) is a security feature that allows you to verify the integrity and publisher of Docker images using digital signatures (via The Update Framework - TUF). When DCT is enabled, Docker verifies that the image you pull has been signed by a trusted publisher and hasn't been tampered with.

```
# Enable Docker Content Trust
export DOCKER_CONTENT_TRUST=1

# Now all pulls, pushes, and builds require signed images
docker pull nginx # Verifies signature
docker pull unsigned-image # FAILS - not signed

# Sign and push an image
docker trust sign myregistry/myapp:v1
docker push myregistry/myapp:v1

# View trust data for an image
docker trust inspect --pretty nginx

# Disable DCT temporarily
DOCKER_CONTENT_TRUST=0 docker pull someimage
```

Q78. What are Docker Secrets?

Docker Secrets is a Swarm feature that provides a secure mechanism for storing and delivering sensitive data (passwords, API keys, TLS certificates, SSH keys) to Docker services. Secrets are encrypted at rest and in transit, and are only available to services that have been granted explicit access.

Key Properties:

- Encrypted using AES-256-GCM at rest on Manager nodes
- Transmitted over TLS-encrypted Swarm connections
- Mounted as files in `/run/secrets/<secret_name>` inside container
- Only services with explicit access can read the secret
- Never exposed via environment variables or `docker inspect`

```
# Create secret from value
echo "P@ssw0rd123" | docker secret create db_password -

# Create secret from file
docker secret create ssl_cert ./certificate.pem

# List secrets
docker secret ls

# Create service with secret access
docker service create \
  --name myapp \
  --secret db_password \
  myapp:latest

# Inside container: secret available at
cat /run/secrets/db_password

# In docker-compose.yml (Swarm mode)
```

```
secrets:
  db_password:
    external: true
services:
  app:
    secrets:
      - db_password
```

Q79. Why should you avoid running containers as root?

Running containers as root is a significant security risk. If an attacker exploits a vulnerability in your containerized application, they gain root-level access inside the container. Due to kernel sharing between containers and the host, this can potentially lead to host compromise or attacks on other containers.

Risks of Running as Root:

- Container escape: Exploitable vulnerabilities could allow an attacker to gain host root access
- Privilege escalation: Root in container has more powers than necessary
- File permission issues: Running as root can corrupt host files via bind mounts
- Fails security compliance: PCI DSS, HIPAA, and SOC2 require non-root execution

How to Run as Non-Root:

```
# In Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

# Create non-root user and set ownership
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
RUN chown -R appuser:appgroup /app

# Switch to non-root user
USER appuser
CMD ["node", "server.js"]

# Verify: check what user runs inside container
docker exec mycontainer whoami
```

Q80. What is AppArmor and Seccomp in Docker?

Seccomp (Secure Computing Mode):

Seccomp is a Linux kernel security feature that restricts which system calls (syscalls) a process can make. Docker applies a default Seccomp profile that blocks ~44 dangerous syscalls, like creating new network namespaces or loading kernel modules.

```
# Run with default seccomp profile (applied automatically)
docker run myapp

# Run with custom seccomp profile
docker run --security-opt seccomp=/path/to/profile.json myapp

# Disable seccomp (not recommended)
docker run --security-opt seccomp=unconfined myapp
```

AppArmor:

AppArmor is a Linux Mandatory Access Control (MAC) security module that defines profiles limiting what resources (files, networks, capabilities) a process can access. Docker supports AppArmor profiles for additional container confinement.

```
# Run with specific AppArmor profile
docker run --security-opt apparmor=docker-default myapp

# Run with custom AppArmor profile
docker run --security-opt apparmor=my-custom-profile myapp

# Disable AppArmor (not recommended)
docker run --security-opt apparmor=unconfined myapp
```

Q81. How do you scan Docker images for vulnerabilities?

1. Docker Scout (Built-in):

```
# Scan image for CVEs
docker scout cves myimage:latest

# Quick overview
docker scout quickview myimage:latest

# Recommendations
docker scout recommendations myimage:latest
```

2. Trivy (Open Source — Most Popular):

```
# Install trivy
brew install aquasecurity/trivy/trivy

# Scan a local image
trivy image myimage:latest

# Scan with severity filter
trivy image --severity HIGH,CRITICAL myimage:latest

# Scan and output as JSON
trivy image -f json -o report.json myimage:latest
```

3. Snyk:

```
snyk container test myimage:latest
```

4. Docker Hub Vulnerability Scanning:

Docker Hub automatically scans images in private repositories and official images. View scan results in the Docker Hub UI under your repository's Vulnerabilities tab.

TOPIC 10 | Advanced & Miscellaneous

Q82. What is the difference between Docker Image ID and Image Tag?

Aspect	Image ID	Image Tag
What it is	SHA256 content hash of the image	Human-readable label/version string
Format	sha256:abc123def456... (64 chars)	name:version (e.g., nginx:1.25)
Uniqueness	Always unique per image	Can be reassigned (e.g., :latest changes)
Mutability	Immutable — fixed forever	Mutable — latest tag changes on new push
Use in scripts	More reliable for exact version	Easier for humans to read

```
docker images
# REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
# nginx         latest    a6bd71f48f68   2 days ago    187MB
# myapp         v1        b2c3d4e5f6a7   1 week ago    350MB

# Pull by tag
docker pull nginx:1.25

# Pull by digest (most reliable for reproducibility)
docker pull nginx@sha256:abc123...

# Tag an image
docker tag nginx:1.25 myregistry/nginx:production
```

Q83. What is Docker Context?

Docker Context allows you to manage and switch between multiple Docker endpoints (local Docker, remote Docker hosts, Swarm clusters, Kubernetes clusters) from a single Docker CLI. Instead of changing your `DOCKER_HOST` environment variable, you can create named contexts and switch between them with a single command.

```
# List contexts
docker context ls

# Create context for remote host
docker context create prod-server \
  --docker "host=ssh://user@prod.example.com"

# Switch to remote context
docker context use prod-server

# Now all docker commands go to remote host
```

```
docker ps

# Switch back to local
docker context use default

# Run single command on specific context
docker --context prod-server ps

# Remove context
docker context rm prod-server
```

Q84. What is BuildKit in Docker?

BuildKit is a modern, improved image build engine for Docker that was introduced in Docker 18.09 and became the default in Docker 23.0. It offers significant improvements over the legacy builder in speed, security, and flexibility.

Key BuildKit Improvements:

- Parallel build stages: Executes independent stages simultaneously
- Better caching: More intelligent layer caching including remote caches
- Secret mounting: Securely pass secrets during build (not stored in layers)
- SSH forwarding: Securely use SSH keys during build
- Pruning control: Better build cache management
- Faster builds: Skips unused stages automatically

```
# Enable BuildKit (Docker < 23.0)
DOCKER_BUILDKIT=1 docker build -t myapp .

# Enable globally in daemon.json
{ "features": { "buildkit": true } }

# BuildKit: Mount secret securely (NOT stored in image)
# In Dockerfile
RUN --mount=type=secret,id=api_key cat /run/secrets/api_key

# Build with secret
docker build --secret id=api_key,src=./api_key.txt .

# BuildKit: SSH forwarding for private repos
docker build --ssh default .
```

Q85. How do you monitor Docker containers?

Built-in Docker Monitoring:

```
# Live resource stats for ALL containers
docker stats

# Stats for specific container
docker stats mycontainer

# One-time snapshot (no streaming)
docker stats --no-stream

# Custom format stats
docker stats --format "{{.Container}}: CPU {{.CPUPerc}} MEM {{.MemUsage}}"

# View container logs
docker logs -f mycontainer
docker logs --tail 100 --since 30m mycontainer

# View container processes
docker top mycontainer

# Docker events stream
docker events
docker events --filter type=container --filter event=die
```

Production Monitoring Stack:

Tool	Purpose
cAdvisor + Prometheus + Grafana	Industry standard stack: collect, store, and visualize container metrics
Datadog / New Relic	Commercial APM and infrastructure monitoring with Docker agents
ELK Stack	Elasticsearch + Logstash + Kibana for centralized log management
Portainer	Web UI for Docker management and basic monitoring
Sysdig	Deep container visibility and security monitoring

Q86. What is docker inspect and when do you use it?

docker inspect returns detailed low-level information about Docker objects (containers, images, volumes, networks) in JSON format. It is one of the most powerful debugging tools available.

```
# Inspect a container
docker inspect mycontainer

# Inspect an image
docker image inspect nginx:latest
```

```
# Inspect a network
docker network inspect mynetwork

# Inspect a volume
docker volume inspect myvolume

# Extract specific fields with Go template
docker inspect -f '{{.NetworkSettings.IPAddress}}' mycontainer
docker inspect -f '{{.State.Status}}' mycontainer
docker inspect -f '{{.State.ExitCode}}' mycontainer
docker inspect -f '{{.Config.Env}}' mycontainer
docker inspect -f '{{.HostConfig.Memory}}' mycontainer

# Get IP of container on specific network
docker inspect -f '{{.NetworkSettings.Networks.mynetwork.IPAddress}}'
mycontainer

# Get all mounted volumes
docker inspect -f '{{json .Mounts}}' mycontainer | python3 -m json.tool
```

Q87. What is docker system prune?

docker system prune is a cleanup command that removes all unused Docker objects to free up disk space. It removes stopped containers, dangling images, unused networks, and optionally unused volumes.

```
# Check disk usage first
docker system df
docker system df -v    # verbose (per-image, per-volume breakdown)

# Remove: stopped containers + dangling images + unused networks
docker system prune

# Also include unused volumes
docker system prune --volumes

# Remove ALL unused images (not just dangling)
docker system prune -a

# Skip confirmation
docker system prune -f

# Most aggressive: everything + volumes, no prompt
docker system prune -af --volumes

# Individual component pruning
docker container prune    # Remove stopped containers
docker image prune -a     # Remove unused images
docker network prune      # Remove unused networks
docker volume prune       # Remove unused volumes
docker builder prune      # Remove build cache
```

Q88. How does Docker handle logging?

Docker captures logs from container stdout and stderr streams and routes them through a configurable logging driver. The default driver is json-file, which stores logs as JSON files on the host.

Default Logging (json-file):

```
docker logs mycontainer
docker logs -f mycontainer      # Follow
docker logs --tail 50 mycontainer # Last 50 lines
docker logs --since 1h mycontainer # Last 1 hour
docker logs -t mycontainer      # With timestamps
```

Logging Drivers:

Driver	Description
json-file (default)	Logs stored as JSON files on host. Viewable with docker logs.
syslog	Send to syslog daemon on host
journald	Send to systemd journal
gelf	Send to Graylog Extended Log Format endpoint
fluentd	Send to Fluentd log collector
awslogs	Send to AWS CloudWatch Logs
splunk	Send to Splunk HTTP Event Collector
none	Disable logging completely

```
# Set log driver at runtime
docker run --log-driver json-file --log-opt max-size=10m --log-opt max-file=3 myapp

# Set in daemon.json globally
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m",
    "max-file": "5"
  }
}

# Send to AWS CloudWatch
docker run --log-driver awslogs \
  --log-opt awslogs-region=us-east-1 \
  --log-opt awslogs-group=my-log-group \
  myapp
```

Q89. What is the difference between Docker and Podman?

Feature	Docker	Podman
Architecture	Client-Server (requires running daemon)	Daemonless — each command is standalone fork/exec
Root privilege	Daemon runs as root by default	Rootless containers by default
Security	Less secure (daemon has root)	More secure (no root daemon)
Compose support	docker-compose / docker compose	podman-compose (mostly compatible)
CLI compatibility	Reference implementation	Docker-compatible (drop-in replacement)
Pods	Not native	Supports Kubernetes-style pods natively
systemd integration	Workaround needed	Native systemd service generation
Developer	Docker Inc.	Red Hat / Open Source Community
K8s output	Not native	podman generate kube → YAML for Kubernetes

```
# Podman is a drop-in replacement
alias docker=podman

# All Docker commands work
podman run -d nginx
podman ps
podman images
podman build -t myapp .
```

Q90. How do you troubleshoot a Docker container that keeps crashing?

Step-by-Step Troubleshooting:

Step 1: Check container status and exit code

```
docker ps -a
# Look at STATUS: Exited (1), Exited (137), OOMKilled, etc.
```

Step 2: View container logs (FIRST thing to check)

```
docker logs mycontainer
docker logs --tail 100 mycontainer
docker logs --since 10m mycontainer
```

Step 3: Inspect the container

```
docker inspect mycontainer
docker inspect -f '{{.State.ExitCode}}' mycontainer
docker inspect -f '{{.State.Error}}' mycontainer
docker inspect -f '{{.State.OOMKilled}}' mycontainer
```

Step 4: Run interactively to debug

```
# Override CMD to get a shell
docker run -it --entrypoint bash myimage
docker run -it --entrypoint sh myimage
```

Step 5: Check Docker events

```
docker events --filter =mycontainer
```

Step 6: Validate Dockerfile

```
docker run -it myimage /bin/sh -c 'ls -la && env && id'
```

Common Exit Codes Reference:

Exit Code	Signal	Meaning
0	—	Success — container exited normally
1	—	App error — check application logs
125	—	Docker daemon error
126	—	Command found but not executable
127	—	Command not found inside container
137	SIGKILL	OOM Killer killed it (out of memory) or docker kill
139	SIGSEGV	Segmentation fault (memory access error)
143	SIGTERM	Graceful shutdown via docker stop
255	—	Exit status out of range

QUICK REFERENCE: TOP DOCKER COMMANDS

Category	Command	Description
Image	<code>docker images</code>	List all local images
Image	<code>docker pull nginx:latest</code>	Pull image from registry
Image	<code>docker build -t app:v1 .</code>	Build from Dockerfile
Image	<code>docker push user/app:v1</code>	Push to registry
Image	<code>docker rmi nginx</code>	Remove image
Image	<code>docker image prune -a</code>	Remove all unused images
Image	<code>docker image history nginx</code>	View image layers
Image	<code>docker image inspect nginx</code>	Detailed image info
Container	<code>docker run -d -p 80:80 nginx</code>	Run in background with port
Container	<code>docker ps</code>	List running containers
Container	<code>docker ps -a</code>	List ALL containers
Container	<code>docker stop myapp</code>	Graceful stop (SIGTERM)
Container	<code>docker kill myapp</code>	Force stop (SIGKILL)
Container	<code>docker rm myapp</code>	Remove stopped container
Container	<code>docker rm -f myapp</code>	Force remove running container
Container	<code>docker exec -it app bash</code>	Interactive shell in container
Container	<code>docker logs -f myapp</code>	Follow container logs
Container	<code>docker stats</code>	Live resource usage
Container	<code>docker inspect myapp</code>	Detailed container info
Container	<code>docker cp myapp:/path ./</code>	Copy file from container
Volume	<code>docker volume create mydata</code>	Create named volume
Volume	<code>docker volume ls</code>	List volumes
Volume	<code>docker volume inspect mydata</code>	Volume details
Volume	<code>docker volume prune</code>	Remove unused volumes
Network	<code>docker network ls</code>	List networks
Network	<code>docker network create mynet</code>	Create network
Network	<code>docker network inspect bridge</code>	Network details
Network	<code>docker network prune</code>	Remove unused networks
System	<code>docker system df</code>	Show disk usage
System	<code>docker system df -v</code>	Verbose disk usage

System	<code>docker system prune -af</code>	Remove all unused objects
System	<code>docker info</code>	System-wide Docker info
System	<code>docker version</code>	Client and server version
System	<code>docker events</code>	Real-time events stream
Compose	<code>docker-compose up -d</code>	Start all services
Compose	<code>docker-compose down</code>	Stop and remove
Compose	<code>docker-compose ps</code>	List service containers
Compose	<code>docker-compose logs -f</code>	Follow all logs
Compose	<code>docker-compose exec svc bash</code>	Shell into service
Swarm	<code>docker swarm init</code>	Initialize Swarm
Swarm	<code>docker node ls</code>	List swarm nodes
Swarm	<code>docker service create --replicas 3 nginx</code>	Deploy service
Swarm	<code>docker service scale web=5</code>	Scale service
Swarm	<code>docker stack deploy -c file.yml stack</code>	Deploy stack

Practice all commands hands-on. Understanding > Memorization.

—— **Connect with the Author** ——

© Sudarshan Gawande | sudarshangawande.com